DS605 · Fundamentals of Machine Learning

# Dimensionality Reduction

*From eigenvectors and SVD to PCA, Kernel PCA, and t-SNE*

Lecture notes for Lecture 20–21 · Arpit Rana · 29 September 2026

## How to Use These Notes

These notes follow the slides for Lecture 20–21. Each topic is explained in three ways:

- **Intuition** — what the idea means, usually with a picture.
- **Mathematics** — the idea written precisely and derived step by step.
- **Implementation** — how to do it in Python with NumPy and scikit-learn.

One small example runs through Sections 2 to 5: five ShopKart customers, each with two features. Every number in those sections can be checked by hand, so please work through the calculations with a pen and paper.

**Notation.** Lower-case letters such as x and u are vectors; upper-case letters such as X and C are matrices. A tilde ($\tilde{x}$, $\tilde{X}$) means the data has been centered. The symbol $^T$ means transpose, so $u^T x$ is the dot product of u and x. $\|u\|$ is the length of u.

The interactive slides let you adjust sliders for most figures in these notes. The figures here are fixed snapshots of the same ideas.

### Contents

1. Why reduce dimensions?
2. Mathematics from scratch: projection, variance, and covariance
3. Eigen-decomposition
4. Singular Value Decomposition (SVD)
5. PCA in practice
6. Kernel PCA
7. t-SNE
8. Summary, exercises, and glossary

# 1. Why Reduce Dimensions?

## 1.1 The problem: ShopKart has too many columns

ShopKart stores about **600 metrics for every customer: visits per month, spend across 40 product categories, clicks on 300 product pages, time spent in** the app for each hour of the day, returns, ratings, and more. Each customer is therefore a point in a 600-dimensional space.

The marketing team makes three requests:

1. **"Show me our customers on one screen."** A screen has 2 dimensions; the data has 600.
2. **"Group customers who behave alike."** Clustering depends on distances, and distances behave badly in 600 dimensions (Section 1.3).
3. **"Predict who will stop buying."** A churn model built on 600 noisy, correlated columns is slow and prone to overfitting.

All three requests lead to the same question: *can we describe each customer with far fewer numbers, without losing what matters?*

## 1.2 Intuition: a good shadow keeps the shape

Hold an object in front of a lamp. Its shadow on the wall is a lower-dimensional picture of the object. From one angle, the shadow reveals almost everything about the shape; from another, it reveals almost nothing.

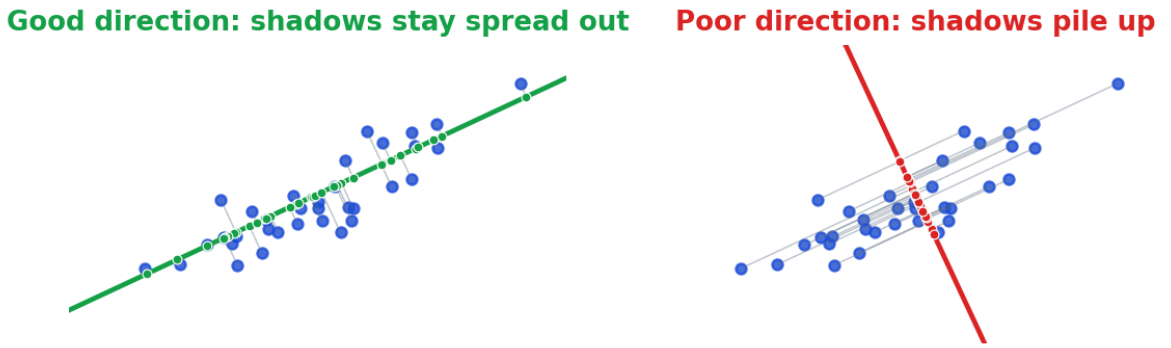



**Figure 1.** The same 2-D point cloud projected onto two different lines. On the left, the projected points (the "shadows") stay spread out; on the right, they pile up, and different points look the same.

> **Key idea**
> Dimensionality reduction is the search for the most informative shadow of the data.

## 1.3 The curse of dimensionality

The **curse of dimensionality** refers to a group of problems that arise when the number of features is large. Three of them matter most for us.

### (a) Space grows much faster than data

Suppose we divide each feature axis into 10 equal parts. The number of cells is $10^d$. To have even one example in every cell, we need at least that many examples.

**Table 1.** How many examples are needed to fill a grid?

| features d | cells $10^d$ | what it feels like |
|---|---|---|
| 1 | 10 | Ten customers are enough |
| 2 | 100 | a small class list |
| 3 | 1,000 | one town's customers |
| 10 | 10,000,000,000 | more than all people on Earth |
| 600 | $10^{600}$ | ShopKart has only about 2 million customers |

In high dimensions, almost every cell is empty. The data becomes **sparse**, and a model has too little evidence in each region to learn from.

### (b) Distances stop meaning much

Take 200 random points in a unit cube and compute all distances between pairs. In 2 dimensions, the distances range from very small to very large. As the number of dimensions grows, all distances crowd around the same value: the nearest point becomes almost as far away as the farthest point.

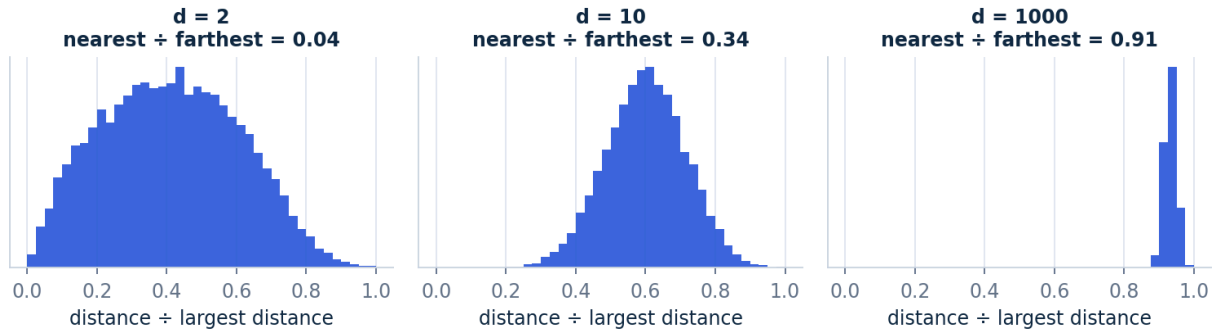



**Figure 2.** Pairwise distances between 200 random points, divided by the largest distance, for d = 2, 10, and 1000. In 1000 dimensions, the nearest neighbor is on average 91% as far away as the farthest point.

Methods that depend on "who is near whom", such as k-nearest neighbors and clustering, lose their power when distances become so similar.

### (c) More features help, then hurt

We might expect that adding features always gives the model more information. In practice, with a **fixed number of training examples**, performance first rises, then peaks, and then falls. This is called the **Hughes phenomenon**. The reason is that the number of examples needed per feature grows exponentially with the number of features, so extra features soon add more noise than signal.

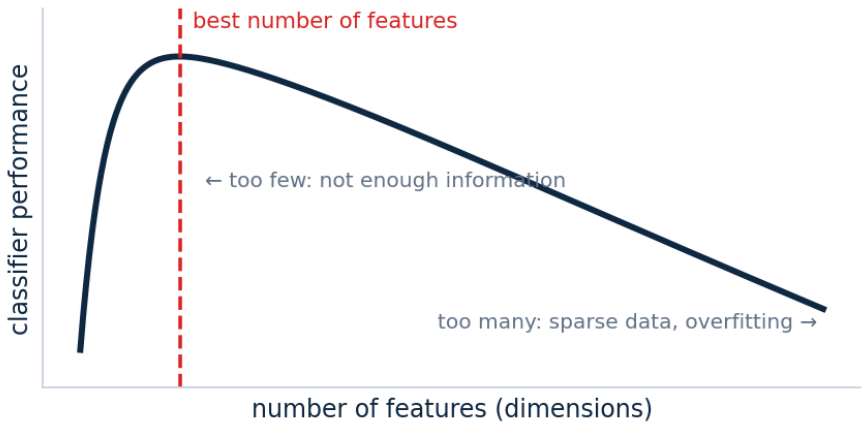



**Figure 3.** Illustrative curve of classifier performance against the number of features. The exact peak depends on the data and the model.

## 1.4 Other reasons to reduce dimensions

- **Speed and memory:** fewer columns mean faster training and smaller models.
- **Visualization:** we can only look at 2 or 3 dimensions at a time.
- **Less overfitting:** fewer inputs leave the model fewer ways to memorize noise.
- **Multicollinearity:** features such as `total_spend` and `spend_per_visit × visits` carry the same information. The new features built by PCA are uncorrelated.
- **Noise filtering:** small, random variation is often dropped, which can improve overall performance.
- **Knowledge discovery:** learning which directions carry information tells us something about the business itself.

## 1.5 Definition

> **Definition**
> Dimensionality reduction is the **transformation** of data from a high-dimensional space to a **low-dimensional space** while **preserving** as much of the original data's meaningful **structure** as possible.

- **Transformation:** a rule f that maps each example $x \in \mathbb{R}^d$ to $z = f(x) \in \mathbb{R}^k$.
- **Low-dimensional:** $k < d$, and usually $k < d$. For ShopKart: 600 → 20.
- **Preserving:** some information is always lost; we try to lose the least useful part.
- **Structure:** PCA keeps variance; Kernel PCA keeps curved patterns; t-SNE keeps neighbors.

## 1.6 Where it sits in a project

Dimensionality reduction is usually a **pre-processing step**: original data (n × d) → dimensionality reduction → new data (n × k) → learner. We lose a little information, and in many (not all) cases, this is more than repaid by a model that is faster and generalizes better.

> **Remember**
> Fit the reducer on the **training data only**, then apply it to the validation and test data. This is the same rule we follow for a scaler. Fitting on test data leaks information and makes the test score too optimistic.

## 1.7 Two strategies

**Table 2.** Feature selection versus feature extraction. Today, we study the three methods in bold.

| | Feature selection | Feature extraction |
|---|---|---|
| idea | keep a subset of the **original** columns | build **new** columns from all the old ones |
| methods | filters (information gain, Gini, correlation); wrappers (try subsets); embedded (Lasso, Lecture 16–17) | linear: **PCA**, LDA; non-linear: **Kernel PCA**, **t-SNE**, UMAP |
| result | easy to explain: the columns keep their names | Often more compact, but new columns are mixtures |

## 1.8 The building block: a new feature is a weighted sum

A ShopKart customer buys from the global store and owes money in two currencies. Suppose 1 US$ = ₹85 and 1 £ = ₹110. We can replace the two columns with one:

| customer | US$ | £ | total in ₹ |
|---|---|---|---|
| P | 5 | 15 | 85·5 + 110·15 = 2,075 |
| Q | 15 | 20 | 85·15 + 110·20 = 3,475 |
| R | 20 | 35 | 85·20 + 110·35 = 5,550 |

$$z = w_1 x_1 + w_2 x_2 = w^T x, \quad \text{with } w = (85, 110)$$

This is a **linear transformation**. PCA builds new features in exactly this way. The difference is that PCA chooses the weights w itself, from the data.

# 2. Mathematics from Scratch

## 2.1 Running example: five ShopKart customers

To keep every number checkable by hand, we use only two features for five customers. Our goal for Sections 2 to 5 is to replace these **two** numbers per customer with **a single number**, while losing as little as possible.

**Table 4.** The running example.

| customer | $x_1$ visits per month | $x_2$ spend (₹ thousand) |
|---|---|---|
| A | 3 | 2 |
| B | 5 | 4 |
| C | 6 | 4 |
| D | 7 | 6 |
| E | 9 | 4 |

## 2.2 Projection onto a direction

A **direction** is a unit vector u, so $\|u\| = 1$. The **projection** of a point x onto the line through u is the number

$$z = u^T x = u_1 x_1 + u_2 x_2$$

z is the length of the "shadow" of x on the line. From z, we can rebuild an approximation of x, and the part we could not rebuild is the error e:

$$\hat{x} = z\,u, \quad e = x - \hat{x}$$

The error e is at a right angle to u. Therefore, Pythagoras gives $\|x\|^2 = z^2 + \|e\|^2$. This small fact is the key to Section 2.5.

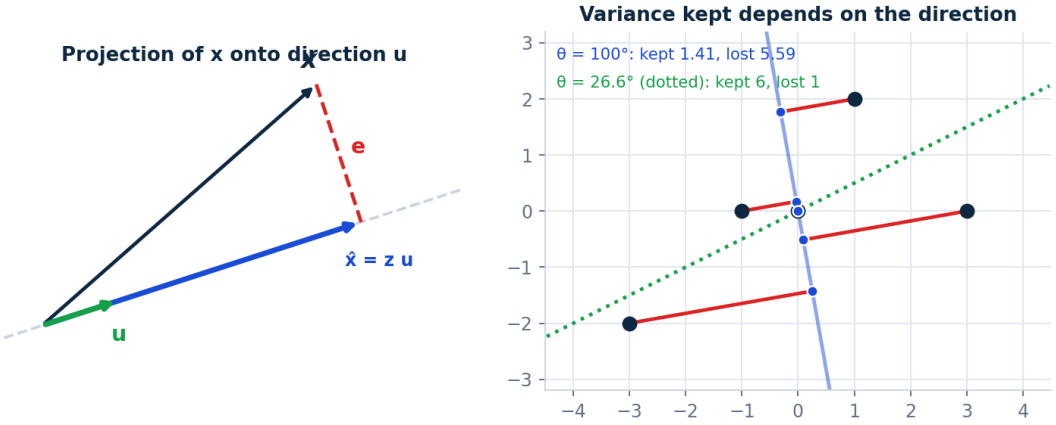



**Figure 4.** Left: projection of x onto a unit vector u, with the rebuilt point $\hat{x} = z\,u$ and the error e. Right: the centered ShopKart customers projected onto a line at 100° (solid) — the red errors are long. The dotted line at 26.6° has much more variance.

## 2.3 Mean, centering, and variance

The **mean** of the data is $\mu = (1/n)\,\Sigma_i\, x_i$. For ShopKart, $\mu = (6, 4)$. **Centering** moves the cloud so that its mean sits at the origin: $\tilde{x}_i = x_i - \mu$. PCA directions pass through the center of the data, so we always center first.

The **sample variance** of feature j measures its spread:

$$s_j^2 = (1/(n-1))\,\Sigma_i\, \tilde{x}_{ij}^2$$

**Table 5.** Centered ShopKart data and the sums we need.

| customer | $\tilde{x}_1$ | $\tilde{x}_2$ | $\tilde{x}_1^2$ | $\tilde{x}_2^2$ | $\tilde{x}_1 \cdot \tilde{x}_2$ |
|---|---|---|---|---|---|
| A | −3 | −2 | 9 | 4 | 6 |
| B | −1 | 0 | 1 | 0 | 0 |
| C | 0 | 0 | 0 | 0 | 0 |
| D | 1 | 2 | 1 | 4 | 2 |
| E | 3 | 0 | 9 | 0 | 0 |
| **sum** | **0** | **0** | **20** | **8** | **8** |

So $s_1^2 = 20/4 = 5$ (visits) and $s_2^2 = 8/4 = 2$ (spend). We divide by $n - 1 = 4$, the sample variance.

> **Remember**
> `np.var` divides by n by default, but `np.cov` divides by n − 1. Mixing the two is a common source of small differences.

## 2.4 Covariance and the covariance matrix

**Covariance** measures whether two features move together. It is positive when both rise together, negative when one rises while the other falls, and near zero when there is no linear link.

$$s_{12} = (1/(n-1))\,\Sigma_i\, \tilde{x}_{i1}\,\tilde{x}_{i2} = (6 + 0 + 0 + 2 + 0)/4 = 2$$

We collect all variances and covariances in the **covariance matrix**:

$$C = (1/(n-1))\,\tilde{X}^T\tilde{X} = [\,5\;\;2\,;\,2\;\;2\,]$$

(Here "[ a b; c d ]" means a 2 × 2 matrix with rows (a, b) and (c, d).) The diagonal holds the variances, the off-diagonal entries hold the covariances, and C is **symmetric**. The **total variance** is the trace: 5 + 2 = 7. For the real ShopKart data, C would be a 600 × 600 matrix.

## 2.5 The key question: which direction keeps the most variance?

If we keep only one number per customer, we should choose the direction u along which the customers are most spread out. In the slides, you can turn a line and watch the variance change; the best angle is about 26.6°, where 6 of the 7 units of variance are kept.

Why is "keep the most variance" the right goal? For each centered customer, Pythagoras's theorem states that $\|\tilde{x}_i\|^2 = z_i^2 + \|e_i\|^2$. Adding over all customers and dividing by n − 1:

$$\text{total variance } (7) \;=\; \text{Var}(z) \text{ (kept)} \;+\; (1/(n-1))\,\Sigma_i \,\|e_i\|^2 \text{ (lost)}$$

> **Key idea**
> The total variance does not depend on the direction. So **keeping the most variance** and **minimizing reconstruction error** are the same goal. Two views, one answer.

## 2.6 The variance of a projection is $u^TCu$

$$\text{Var}(z) = (1/(n-1))\,\Sigma_i\,(u^T\tilde{x}_i)^2 = u^T\,[\,(1/(n-1))\,\Sigma_i\,\tilde{x}_i\tilde{x}_i^T\,]\,u = u^T C\,u$$

Two checks with ShopKart:

- u = (1, 0), the visits axis: $u^TCu = C_{11} = 5$, the variance of visits. Correct.
- u = (2, 1)/√5, the best angle: $u^TCu = (1/5)(5\cdot 2^2 + 2\cdot(2\cdot 2\cdot 1) + 2\cdot 1^2) = 30/5 = 6$. Correct.

The whole dataset is now summarised in a single small matrix C. The problem is to find the unit vector u that maximizes $u^TCu$.

## 2.7 Optimization leads to eigenvectors

$$\text{maximize } u^TCu \quad \text{subject to} \quad u^Tu = 1$$

1. Use a Lagrange multiplier λ for the constraint: $L(u, \lambda) = u^TCu - \lambda(u^Tu - 1)$.
2. Set the gradient to zero: $\partial L/\partial u = 2Cu - 2\lambda u = 0$.
3. This gives the eigenvalue equation **Cu = λu**.
4. Multiply both sides by $u^T$: $u^TCu = \lambda\, u^Tu = \lambda$.

> **Key idea**
> The best direction is an **eigenvector** of C, and the variance it keeps is its **eigenvalue** λ. To preserve the most variance, take the eigenvector corresponding to the largest eigenvalue.

# 3. Eigen-Decomposition

## 3.1 Intuition: eigenvectors only stretch

A matrix is a machine that moves vectors. When we multiply our C by a vector v, the result Cv usually points in a **different** direction. For a few special directions, Cv points the **same** way as v; it is only stretched (or shrunk). These are the **eigenvectors**, and the stretch factor is the **eigenvalue**: Cv = λv. "Eigen" is German for "own": these are the matrix's own directions.

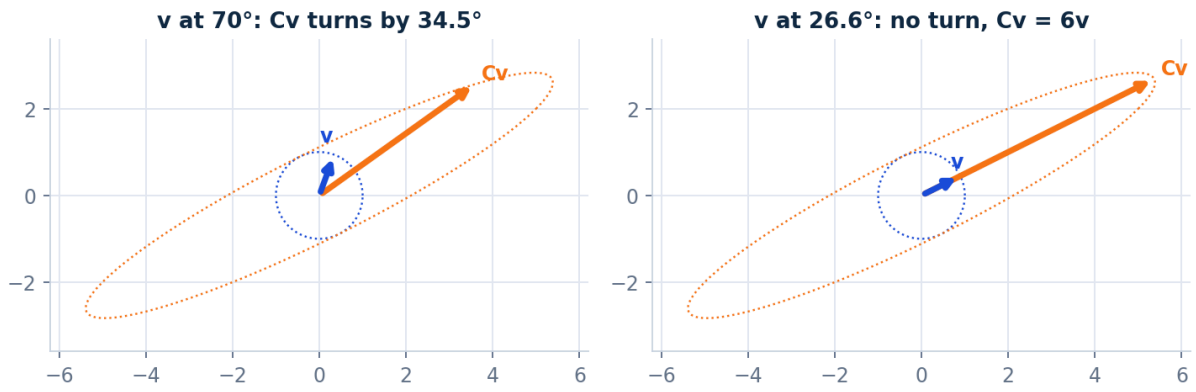



**Figure 5.** Left: a vector v at 70° is turned by 34.5° when multiplied by C. Right: at 26.6° there is no turn, only a stretch by 6. Dotted curves: all unit vectors v, and where C sends them.

## 3.2 Step 1 — eigenvalues from the characteristic equation

Rewrite Cu = λu as (C − λI)u = 0. We want a **non-zero** u. That is possible only when C − λI squashes some direction to zero, that is, when it is singular:

$$\det(C - \lambda I) = 0$$

For ShopKart:

$$\det\begin{bmatrix} 5-\lambda & 2 \\ 2 & 2-\lambda \end{bmatrix} = (5-\lambda)(2-\lambda) - 2\cdot 2 = \lambda^2 - 7\lambda + 6 = (\lambda - 6)(\lambda - 1) = 0$$
$$\lambda_1 = 6, \quad \lambda_2 = 1$$

Two quick checks: the sum 6 + 1 = 7 equals the trace of C, and the product 6 · 1 = 6 equals det(C) = 10 − 4.

## 3.3 Step 2 — eigenvectors

**For $\lambda_1 = 6$:** C − 6I = [ −1  2 ; 2  −4 ]. The first row gives $-u_1 + 2u_2 = 0$, so $u_1 = 2u_2$. Take (2, 1), whose length is √5, and normalize:

$$u_1 = (1/\sqrt{5})(2, 1) = (0.894, 0.447)$$

**For $\lambda_2 = 1$:** C − I = [ 4  2 ; 2  1 ]. The first row gives $2u_1 + u_2 = 0$, so $u_2 = -2u_1$. Take (1, −2) and normalize:

$$u_2 = (1/\sqrt{5})(1, -2) = (0.447, -0.894)$$

Verify: C(2, 1) = (5·2 + 2·1, 2·2 + 2·1) = (12, 6) = 6·(2, 1). Orthogonal: (2)(1) + (1)(−2) = 0.

> **Remember**
> Any multiple of an eigenvector is also an eigenvector. We set the length to 1, but the sign remains free: $-u_1$ is equally correct. Different software may return either sign.

## 3.4 Eigen-decomposition of a symmetric matrix

$$C = Q\,\Lambda\,Q^T = \Sigma\square\ \lambda\square\ u\square u\square^T$$
$$Q = \begin{bmatrix} 0.894 & 0.447 \\ 0.447 & -0.894 \end{bmatrix}, \quad \Lambda = \begin{bmatrix} 6 & 0 \\ 0 & 1 \end{bmatrix}$$

The columns of Q are the eigenvectors, and Λ holds the eigenvalues. The **spectral theorem** guarantees three useful facts for every symmetric matrix, such as a covariance matrix:

1. **Real eigenvalues:** no complex numbers appear.
2. **Orthogonal eigenvectors:** $Q^TQ = I$. The new axes meet at right angles, so the new features are uncorrelated.
3. **Non-negative eigenvalues** (for a covariance matrix): $\lambda_i \geq 0$, because each $\lambda_i$ is a variance.

## 3.5 What the eigenvectors say about ShopKart

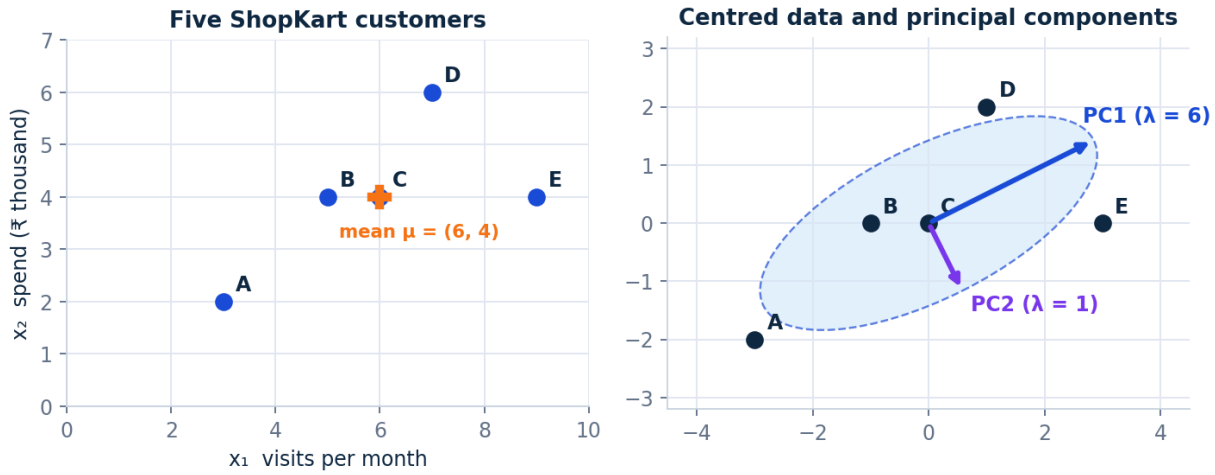



**Figure 6.** Left: the five customers and their means. Right: centered data with the covariance ellipse. The ellipse axes follow the eigenvectors, and their lengths grow with $\sqrt{\lambda}$.

**Table 6.** Principal components of the ShopKart example.

| | direction | variance λ | share of variance | meaning |
|---|---|---|---|---|
| PC1 | (0.894, 0.447) | 6 | 6/7 = 85.7% | "engagement": visits and spend grow together |
| PC2 | (0.447, −0.894) | 1 | 1/7 = 14.3% | "browsing vs buying": many visits but low spend gives a positive score (a window-shopper) |

# 4. Singular Value Decomposition (SVD)

## 4.1 Why not always use the covariance matrix?

1. **Size.** A 1000 × 1000 photo has $d = 10^6$ pixels. Its covariance matrix has $10^{12}$ entries, about 8 terabytes.
2. **Accuracy.** Forming $\tilde{X}^T\tilde{X}$ squares every number. A small value such as $10^{-9}$ becomes $10^{-18}$, which is below double-precision accuracy and is lost.
3. **Shape.** Eigen-decomposition needs a square matrix. The data matrix is n × d and almost never square.

**Singular Value Decomposition** works on $\tilde{X}$ directly, for any shape, and is numerically stable. NumPy and scikit-learn compute PCA this way.

## 4.2 Definition

Every real n × d matrix can be written as a product of three simple matrices:

$$X = U \Sigma V^T$$

| matrix | shape | property | name |
|---|---|---|---|
| U | n × n | orthogonal: $U^TU = I$; one row per example | left singular vectors |
| Σ | n × d | diagonal, $\sigma_1 \geq \sigma_2 \geq \ldots \geq 0$ | singular values |
| V | d × d | orthogonal: $V^TV = I$; one column per direction in feature space | right singular vectors |

In practice, we use the **thin** SVD, which keeps only the first r = min(n, d) columns of U (`full_matrices=False` in NumPy).

## 4.3 Intuition: every matrix is a rotation, a stretch, or a rotation

Think of a 2 × 2 matrix $M = U\Sigma V^T$ acting on the unit circle. $V^T$ first rotates the special directions $v_1$ and $v_2$ onto the axes. Σ then stretches each axis by $\sigma_1$ and $\sigma_2$. Finally, you rotate the result into its final place. The circle always becomes an ellipse whose long axis has length $\sigma_1$ in direction $u_1$.

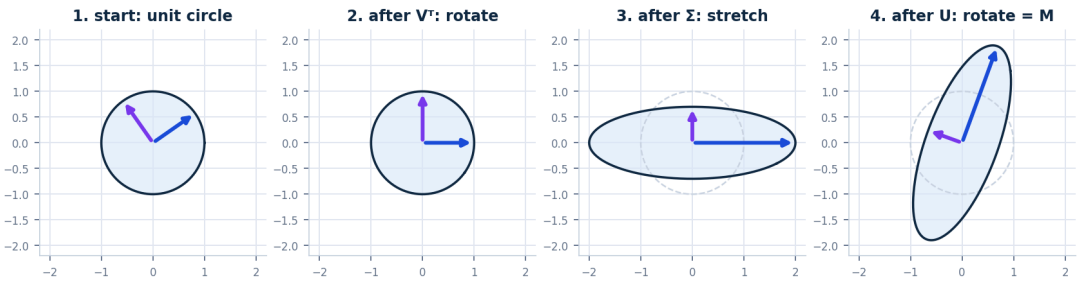



**Figure 7.** The unit circle is transformed step by step by $V^T$ (rotate), Σ (stretch by 2.0 and 0.7), and U (rotate). The blue and violet arrows start as $v_1$ and $v_2$ and end as $\sigma_1 u_1$ and $\sigma_2 u_2$.

### 4.4 SVD gives the principal components directly

$$\tilde{X}^T\tilde{X} = (U\Sigma V^T)^T(U\Sigma V^T) = V\,\Sigma^T\,(U^TU)\,\Sigma\,V^T = V\,\Sigma^2\,V^T$$

Compare this with the eigen-decomposition $C = (1/(n-1))\,\tilde{X}^T\tilde{X} = Q\Lambda Q^T$. Matching the two gives three results:

- **Directions:** $V = Q$. The right singular vectors are the principal components.
- **Variances:** $\lambda_j = \sigma_j^2 / (n - 1)$. We never need to form $C$.
- **Scores:** $Z = \tilde{X}V = U\Sigma$ gives the new coordinates of every example.

### 4.5 Worked example: SVD of the ShopKart data

The centered data matrix has rows (−3, –2), (−1, 0), (0, 0), (1, 2), (3, 0). `np.linalg.svd(Xc)` returns $\sigma = (4.899, 2.000)$ and $V^T$ with rows (0.894, 0.447) and (0.447, −0.894). Converting with $n - 1 = 4$:

$$\lambda_1 = 4.899^2 / 4 = 24 / 4 = 6, \quad \lambda_2 = 2^2 / 4 = 4 / 4 = 1$$

The rows of $V^T$ are exactly $u_1$ and $u_2$ from Section 3, and the variances 6 and 1 match. Same answer, different road.

### 4.6 The best low-rank approximation

$$\tilde{X} = \sum_{i=1}^{r} \sigma_i\, u_i v_i^T \;\approx\; \tilde{X}_k = \sum_{i=1}^{k} \sigma_i\, u_i v_i^T$$

The **Eckart–Young theorem** says that among all matrices of rank k, $\tilde{X}_k$ is the closest to $\tilde{X}$, and its total squared error is exactly the sum of the dropped $\sigma_j^2$ (all $j > k$). For ShopKart with $k = 1$, the squared error is $\sigma_2^2 = 4$ over all five customers. We will see this number again in Section 5.2.

> **Key idea**
> Keeping the top-k singular directions is the best possible way to squeeze the data into k dimensions, when "best" means the smallest squared error. That is PCA.

## 5. PCA in Practice

### 5.1 The PCA algorithm

1. **Center:** $\tilde{X} = X - \mu$. Also, scale the features if their units differ (Section 5.3).
2. **Decompose:** compute the SVD $\tilde{X} = U\Sigma V^T$ (or eigen-decompose $C$).
3. **Order:** sort directions by variance $\lambda_j = \sigma_j^2/(n - 1)$, largest first.
4. **Choose k:** keep enough components, for example, 95% of the variance (Section 5.4).
5. **Project:** $Z = \tilde{X}W_k$, where $W_k$ holds the first k columns of $V$ (a $d \times k$ matrix).
6. **Reconstruct (optional):** $\hat{X} = ZW_k^T + \mu$, an approximation of the original data.

### 5.2 Worked example: two numbers become one (k = 1)

Project each centered customer onto $u_1 = (0.894, 0.447)$, then rebuild it with $\hat{x} = \mu + z_1 u_1$.

**Table 8.** PCA with one component on the ShopKart customers.

| customer | original x | centred $\tilde{x}$ | score $z_1$ | rebuilt $\hat{x}$ | squared error |
|---|---|---|---|---|---|
| A | (3, 2) | (−3, −2) | −3.578 | (2.8, 2.4) | 0.2 |
| B | (5, 4) | (−1, 0) | −0.894 | (5.2, 3.6) | 0.2 |
| C | (6, 4) | (0, 0) | 0.000 | (6.0, 4.0) | 0.0 |
| D | (7, 6) | (1, 2) | 1.789 | (7.6, 4.8) | 1.8 |
| E | (9, 4) | (3, 0) | 2.683 | (8.4, 5.2) | 1.8 |
| **total** | | | | | **4.0** |

For customer A, for example: $z_1 = 0.894 \cdot (-3) + 0.447 \cdot (-2) = -3.578$, and $\hat{x} = (6, 4) + (-3.578)(0.894, 0.447) = (2.8, 2.4)$. The squared error is $(3 - 2.8)^2 + (2 - 2.4)^2 = 0.04 + 0.16 = 0.2$.

- Variance of the scores: $(3.578^2 + 0.894^2 + 0 + 1.789^2 + 2.683^2)/4 = 24/4 = 6$. We kept 6/7 = 85.7% of the variance.
- Total squared error $4 = \sigma_2^2$, exactly as the Eckart–Young theorem promised in Section 4.6.

## 5.3 Units matter: scale before PCA

Suppose we record spending in **rupees** instead of ₹ thousand. The customers are the same; only the unit has changed.

**Table 9.** The same customers with different units.

| spend measured in | covariance matrix | first principal component | what happens |
|---|---|---|---|
| ₹ thousand | [ 5  2 ; 2  2 ] | (0.894, 0.447), 85.7% | both features count |
| ₹ | [ 5  2000 ; 2000  2 000 000 ] | (0.001, 1.000), 99.9998% | PC1 is simply "spend"; visits are ignored |
| standardized | correlation matrix, r = 0.632 | (0.707, 0.707), 81.6% | Each feature has a mean of 0 and a variance of 1 |

> **Remember**
> PCA chases large numbers, not important ones. When features have different units, use `StandardScaler` first. When all features share a single unit (for example, pixel brightness), scaling is usually unnecessary.

## 5.4 Explained variance and choosing k

$$\mathrm{EVR}_k = \lambda_k / \Sigma_j \lambda_j, \quad \mathrm{cumulative}(k) = \mathrm{EVR}_1 + \mathrm{EVR}_2 + \ldots + \mathrm{EVR}_k$$

For ShopKart, EVR = (6/7, 1/7) = (0.857, 0.143). There are four common ways to choose k:

1. **Variance threshold:** the smallest k whose cumulative EVR reaches 95% (or 90%, 99%).
2. **Elbow:** plot EVR against k and stop where the curve bends and flattens.
3. **Visualization:** k = 2 or 3, because that is what we can plot.
4. **As a hyperparameter:** tune k with cross-validation of the final model. When a task is supervised, this is usually the most reliable method.

## 5.5 Example: compressing handwritten digits

scikit-learn's `load_digits` dataset has 1,797 images of 8 × 8 pixels, so d = 64. Figure 8 shows how much variance is retained as k grows, and Figure 9 shows the images reconstructed from k values.

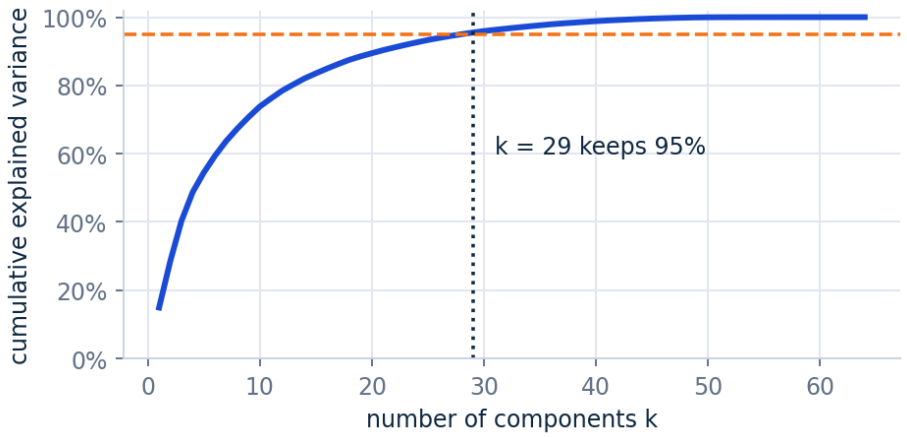



**Figure 8.** Cumulative explained variance for the digits data. 95% of the variance requires k = 29 components (90% requires k = 21).

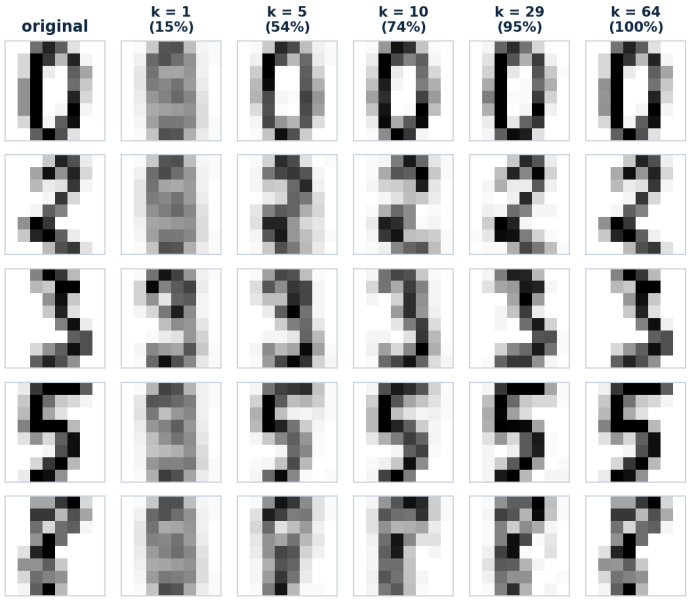



**Figure 9.** Five digits rebuilt from k = 1, 5, 10, 29, and 64 principal components. With k = 10, the digits are already readable; with k = 64, the rebuild is exact.

For MNIST (28 × 28 = 784 pixels), 95% of the variance is captured by about 154 components, roughly 20% of the original dimensionality. The mean squared distance between the original and the rebuilt data is called the **reconstruction error**.

## 5.6 Implementation 1: PCA from scratch with NumPy

```
import numpy as np

X = np.array([[3, 2], [5, 4], [6, 4], [7, 6], [9, 4]], dtype=float)

mu = X.mean(axis=0)                                  # [6. 4.]
Xc = X - mu                                          # 1. centre

U, s, Vt = np.linalg.svd(Xc, full_matrices=False)    # 2. SVD (already sorted)
explained_var = s**2 / (len(X) - 1)                  # 3. [6. 1.]
ratio = explained_var / explained_var.sum()          #    [0.857 0.143]

k = 1                                                # 4. choose k
W = Vt[:k].T                                         #    d x k -> [[0.894], [0.447]]
Z = Xc @ W                                           # 5. scores
X_hat = Z @ W.T + mu                                 # 6. rebuild
print(((X - X_hat)**2).sum())                        #    4.0  (= s[1]**2)
```

Every number in the comments matches the hand calculation in Sections 3 to 5.

## 5.7 Implementation 2: the eigen route, and two traps

```
C = np.cov(Xc, rowvar=False)          # [[5. 2.] [2. 2.]]  (divides by n-1)
vals, vecs = np.linalg.eigh(C)        # eigh: for symmetric matrices
print(vals)                           # [1. 6.]   <- ascending order!
order = np.argsort(vals)[::-1]        # largest first
vals, vecs = vals[order], vecs[:, order]
print(vecs[:, 0])                     # [-0.894 -0.447]   <- sign flipped vs SVD
```

- **Trap 1 — order:** `np.linalg.eigh` returns eigenvalues from smallest to largest. Always sort before choosing k.
- **Trap 2 — sign:** u and −u describe the same line. Libraries (or even different runs) may flip signs. The scores flip, too, but the information is the same.

## 5.8 Implementation 3: scikit-learn

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
Z = pca.fit_transform(X)            # centers the data for you
pca.components_                     # rows = principal components (k x d)
pca.explained_variance_             # [6. 1.]
pca.explained_variance_ratio_       # [0.857 0.143]
X_back = pca.inverse_transform(Z)   # back to d dimensions

pca95 = PCA(n_components=0.95)      # keep 95% of the variance
Z95 = pca95.fit_transform(X_train)
pca95.n_components_                 # the k it chose
```

- `components_` stores the components as **rows**: it equals W□ᵀ, the first k rows of Vᵀ. The first principal component is `pca.components_[0]`.
- For large data, `PCA(svd_solver="randomized")` finds only the top k directions and is much faster when k ≪ d.

- `IncrementalPCA` learns from mini-batches with `partial_fit`, for data that does not fit in memory.

## 5.9 Implementation 4: a ShopKart churn model with PCA

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

pipe = make_pipeline(StandardScaler(),         # units differ: scale first
                     PCA(),
                     LogisticRegression(max_iter=1000))

param_grid = {"pca__n_components": [10, 25, 50, 100, 0.95]}  # k is tuned
grid = GridSearchCV(pipe, param_grid, cv=5, scoring="roc_auc")
grid.fit(X_train, y_train)                     # y = 1 if the customer churned

print(grid.best_params_, grid.score(X_test, y_test))
```

The pipeline fits the scaler and PCA only within each training fold, so no information leaks from the validation data.

## 5.10 When PCA is not enough

1. **Only straight directions.** PCA projects onto flat lines and planes. It cannot unroll data that lies on a curved surface.
2. **Variance is not always what matters.** PCA ignores labels. A low-variance direction may be the one that separates churners from loyal customers. (LDA uses the labels.)
3. **Sensitive to scale and outliers.** One extreme customer can pull a component towards itself.
4. **Harder to explain.** Each component mixes all original features.

The next two methods address point 1: **Kernel PCA** for curved structures and **t-SNE** for faithful 2-D pictures of neighborhoods.

# 6. Kernel PCA

## 6.1 Motivation: a ring around a core

ShopKart plots customers on two behavior scores. **Loyal customers** form a core; **occasional customers** form a ring around them. Linear PCA can only turn the axes. When we project onto PC1, the two groups overlap: no straight line separates a ring from its center.

## 6.2 The idea: lift the data, then cut it flat

Add a third feature, the squared distance from the center:

$$\varphi(x_1, x_2) = (x_1, x_2, {x_1}^2 + {x_2}^2)$$

In this lifted space, the ring rises, and the core stays low, so a **flat plane** separates them. A linear method in the lifted space is a curved method in the original space.

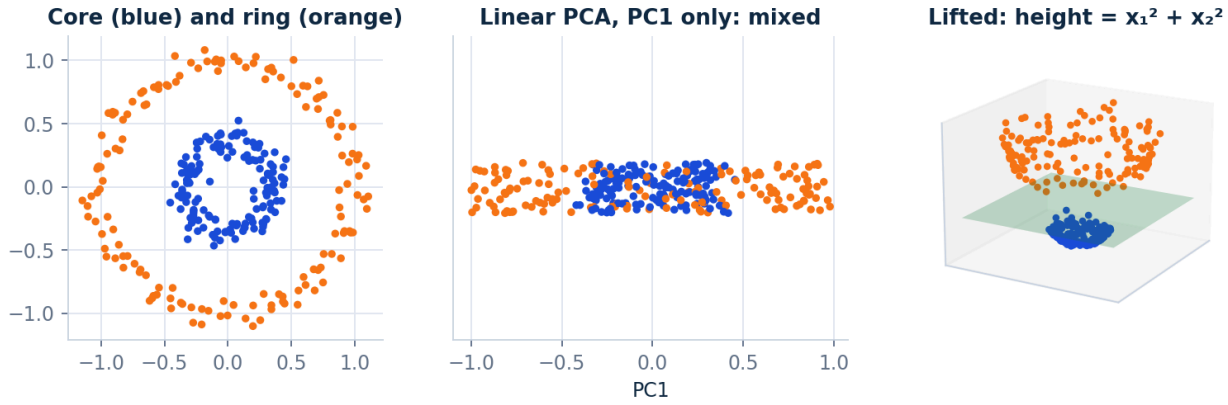



**Figure 10.** Left: the core (blue) and ring (orange). Middle: linear PCA keeps only PC1 and mixes the groups. Right: after lifting with height $x_1^2 + x_2^2$, a flat plane separates them.

## 6.3 The kernel trick

Lifting can be expensive: $\varphi(x)$ may have millions of features, or even infinitely many. However, PCA can be rewritten so that it needs only **dot products** $\varphi(x)^T\varphi(x')$. A **kernel** gives the dot product directly from the original points:

$$k(x, x') = \varphi(x)^T\varphi(x')$$

**Check with numbers.** For the degree-2 polynomial map $\varphi(x) = (x_1^2, \sqrt{2}\cdot x_1x_2, x_2^2)$, take $x = (1, 2)$ and $x' = (3, 1)$.

- Lift first: $\varphi(x) = (1, 2\sqrt{2}, 4)$, $\varphi(x') = (9, 3\sqrt{2}, 1)$, and the dot product is $9 + 12 + 4 = 25$.
- Use the kernel: $(x^Tx')^2 = (3 + 2)^2 = 25$. Same answer, without building $\varphi$.

**Table 10.** Common kernels.

| kernel | k(x, x′) | note |
|---|---|---|
| linear | $x^T x'$ | gives plain PCA |
| polynomial | $(\gamma\, x^T x' + c)^p$ | degree p |
| RBF (Gaussian) | $\exp(-\gamma \lVert x - x' \rVert^2)$ | matches an infinite-dimensional φ |
| sigmoid | $\tanh(\gamma\, x^T x' + c)$ | |

## 6.4 The Kernel PCA algorithm

1. **Kernel matrix:** $K_{i□} = k(x_i, x_□)$. It is n × n: one row and one column per **example**, not per feature.
2. **Center in feature space:** $\tilde{K} = K - 1_□K - K1_□ + 1_□K1_□$, where every entry of the n × n matrix $1_□$ is 1/n. This is "subtract the mean" done without ever computing φ.
3. **Eigen-decompose:** $\tilde{K}a_□ = \lambda_□ a_□$ with unit eigenvectors $a_□$, sorted by $\lambda_□$.
4. **Project:** for training points, $z_{i□} = \sqrt{\lambda_□} \cdot a_{i□}$. For a new point x, $z_□(x) = (1/\sqrt{\lambda_□})\ \Sigma_i\ a_{i□}\ \tilde{k}(x_i, x)$, where $\tilde{k}$ is the centered kernel.

- With the linear kernel, Kernel PCA gives exactly the PCA scores.
- **The price:** K grows with $n^2$. For 100,000 customers, it has $10^{10}$ entries, and the eigen-decomposition costs about $O(n^3)$ operations.

## 6.5 Choosing γ for the RBF kernel

γ controls how quickly similarity fades with distance; the width of the kernel is about $1/\sqrt{\gamma}$. For our ring data:

- **γ too small (0.1):** every point looks similar to every other point. The result is close to linear PCA, and the groups overlap on PC1.
- **γ about right (3 to 5):** the core and the ring are fully separated along PC1 alone.
- **γ too large (10 or more):** each point is similar only to its closest neighbors, and the structure breaks into fragments.

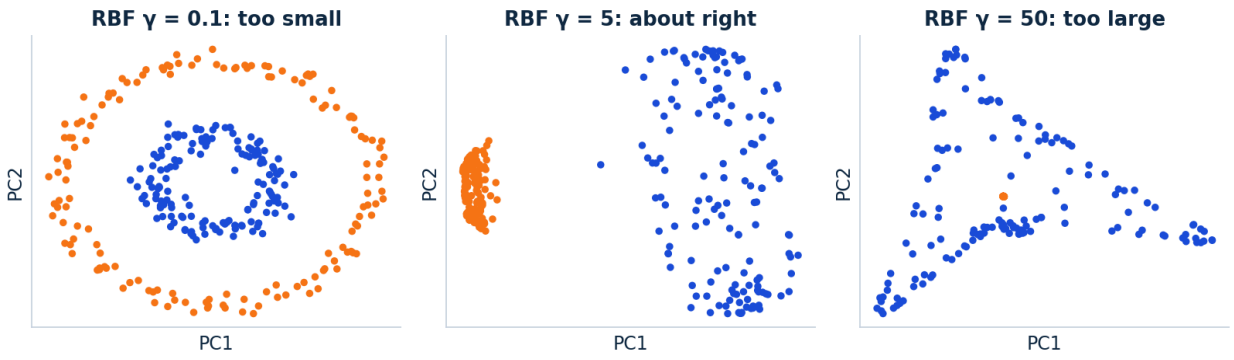



**Figure 11.** Kernel PCA with the RBF kernel on the ring data for three values of γ.

### 6.6 Implementation

```
from sklearn.decomposition import KernelPCA

kpca = KernelPCA(n_components=2, kernel="rbf", gamma=5)
Z = kpca.fit_transform(X)          # X: the ring data (n x 2)
Z_new = kpca.transform(X_new)      # works for new points

# an approximate way back to the original space (a "pre-image")
kpca = KernelPCA(n_components=2, kernel="rbf", gamma=5,
                 fit_inverse_transform=True)
Z = kpca.fit_transform(X)
X_back = kpca.inverse_transform(Z)
```

Kernel PCA is unsupervised, so it has no direct score. We usually tune the kernel and γ through the model that follows it:

```
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

pipe = make_pipeline(KernelPCA(n_components=2), LogisticRegression())
grid = GridSearchCV(pipe, {
    "kernelpca__kernel": ["rbf", "sigmoid"],
    "kernelpca__gamma": [0.1, 1, 3, 5, 10],
}, cv=3)
grid.fit(X, y)
print(grid.best_params_)
```

Without labels, another option is to choose the settings that minimize reconstruction (pre-image) error.

## 7. t-SNE

### 7.1 Motivation: PCA keeps the far, loses the near

The marketing team wants a single picture of all customers, with **similar customers seated together**. PCA keeps the directions of the largest variance. This preserves large distances (the global shape) well, but small distances between neighbors are often crushed together. For the digit data, the best 2-D PCA view keeps only 28.5% of the variance (14.9% + 13.6%), and most digit groups overlap (Figure 14, left).

New goal: keep **who is next to whom**, and let large distances go. This is what **t-SNE** (t-distributed Stochastic Neighbor Embedding) does.

### 7.2 Manifolds: the Swiss roll

Some data lies on a 2-D sheet that is rolled up in 3-D. Such a curved surface is called a **manifold**. A good method "unrolls" it so that points close along the sheet stay close, and points far apart along the sheet stay apart.

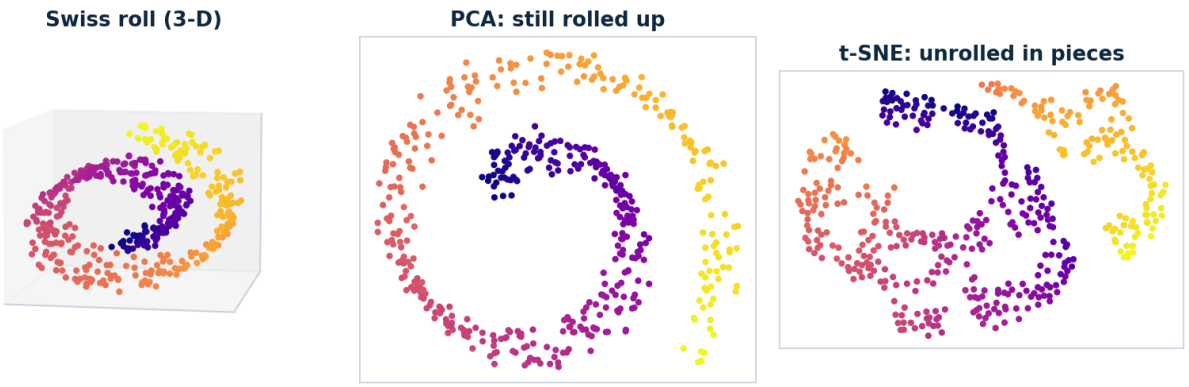



**Figure 12.** Left: the Swiss roll; color shows position along the sheet. Middle: PCA only finds the flattest view, so the sheet stays rolled up. Right: t-SNE keeps neighbors together, and the colors change smoothly (the sheet is unrolled, in pieces).

## 7.3 Step 1: turn distances into probabilities

$$p_{j|i} = \exp(-\|x_i - x_j\|^2 / 2\sigma_i^2) \,/\, \Sigma_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)$$
$$p_{ij} = (p_{j|i} + p_{i|j}) \,/\, 2n$$

- **Read it as:** "if $x_i$ picked a neighbor at random, how likely is it to pick $x_j$?" Near points get a high probability.
- **Perplexity:** each $\sigma_i$ is chosen so that 2^H($P_i$) equals the perplexity we set (often between 5 and 50). H is the entropy from Lecture 11–12. Think of perplexity as the effective number of neighbors.
- **Adapts to density:** in crowded regions, $\sigma_i$ becomes small; in sparse regions, it becomes large, so each point gets a fair set of neighbors.

## 7.4 Step 2: a heavy tail in the map

Give each point a 2-D position $y_i$. Similarity in the map uses a Student-t curve with one degree of freedom:

$$q_{ij} = (1 + \|y_i - y_j\|^2)^{-1} \,/\, \Sigma_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}$$

**Why not a Gaussian again? The crowding problem.** In 64-D, a point can have many neighbors at a moderate distance. A 2-D map has far less room for them. The heavy tail of the t-curve lets moderately far points sit **much** farther away in the map, so separate groups do not squash together. That is the "t" in t-SNE.

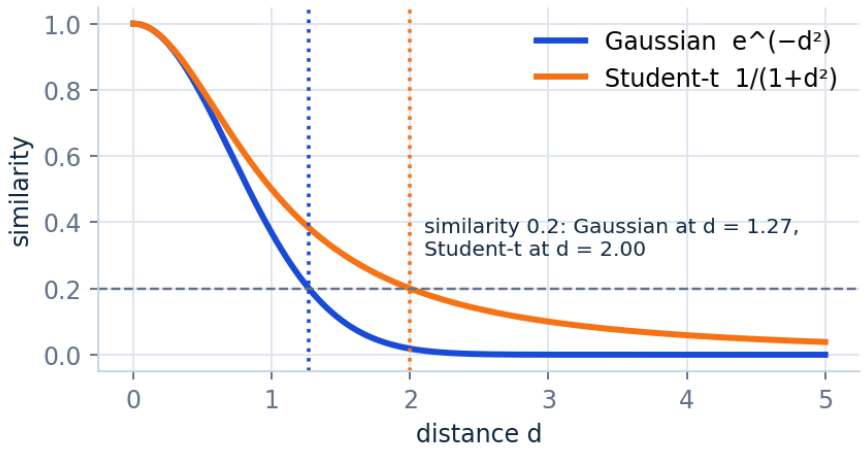



**Figure 13.** Gaussian and Student-t similarity curves. For the same similarity of 0.2, the Student-t curve places the pair at a distance of 2.00 instead of 1.27.

## 7.5 Step 3: Make the map agree using the KL divergence

$$\text{Cost} = \text{D_KL}(P \parallel Q) = \Sigma_{i \neq j} p_{ij} \log(p_{ij} / q_{ij})$$
$$\partial \text{Cost} / \partial y_i = 4 \Sigma_j (p_{ij} - q_{ij})(y_i - y_j) / (1 + \|y_i - y_j\|^2)$$

- **Uneven penalties:** neighbors placed far apart (large p, small q) cost a lot; strangers placed close together (small p, large q) cost little. So t-SNE mainly protects **neighborhoods**. (KL divergence was introduced in Lecture 11–12.)
- **Springs:** if $p_{ij} > q_{ij}$ the pair attracts; if $p_{ij} < q_{ij}$ it repels. The map settles when the forces balance.
- **Optimization:** start from PCA (or at random) and run gradient descent for about 1,000 steps. Early on, $p_{ij}$ is multiplied by a factor ("early exaggeration") so that clusters form first.

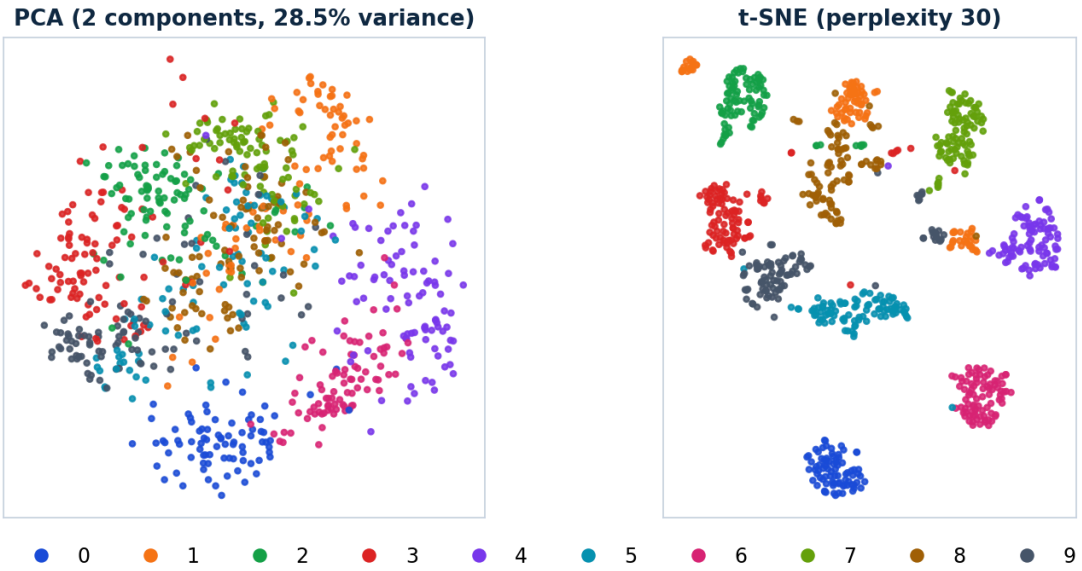



**Figure 14.** The same 900 handwritten digits mapped to 2-D by PCA (left) and t-SNE with perplexity 30 (right), coloured by the true digit. The labels were not used to make either map.

## 7.6 How not to misread a t-SNE plot

- **Cluster size means nothing.** t-SNE expands dense groups and shrinks sparse ones.
- **Gaps mean little.** Distances between clusters are unreliable; only the "these points are neighbors" relationship is reliable.
- **Try several settings.** Perplexity and the random seed change the picture; trust patterns that appear in every run.
- **No transform() for new points.** A new customer cannot be placed on an existing map. t-SNE is for looking, not a pre-processing step for models.
- **Cost.** Exact t-SNE is O(n²); the Barnes–Hut method (scikit-learn's default) is O(n log n).
- **Reduce first.** For very wide data, use PCA to about 50 dimensions first. It is faster and removes noise.

## 7.7 Implementation

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn. manifold import TSNE

X_scaled = StandardScaler().fit_transform(X_customers)     # 600 features
X50 = PCA(n_components=50).fit_transform(X_scaled)         # speed + less noise

tsne = TSNE(n_components=2,
            perplexity=30,        # try 5, 30, 50
            init="pca",           # more stable than a random start
            random_state=42)      # fix the seed to repeat a picture
Y = tsne.fit_transform(X50)       # n x 2 map, only for plotting
print(tsne.kl_divergence_)        # final cost
```

A related method, **UMAP** (package `umap-learn`), is often faster and keeps more of the global layout. It also offers `transform()` for new points.

# 8. Summary, Exercises, and Glossary

## 8.1 Three methods side by side

**Table 11.** Comparison of the three methods.

| | PCA | Kernel PCA | t-SNE |
|---|---|---|---|
| type | linear projection | non-linear, via a kernel | non-linear, probabilistic |
| keeps | largest variance (global shape) | curved structure, set by the kernel | local neighbourhoods |
| core maths | eigen-decomposition / SVD of $\tilde{X}$ | eigen-decomposition of the n × n kernel matrix | KL divergence + gradient descent |
| same result each run | yes (up to sign) | yes (up to sign) | No, depends on the seed |
| new points | `transform()` ✓ | `transform()` ✓ | ✗ |
| main settings | k | k, kernel, γ | perplexity, iterations |
| best use | compression, denoising, pre-processing | pre-processing for curved data | visualisation only |

## 8.2 Key takeaways

1. High-dimensional data is sparse, and its distances lose meaning. Reducing dimensions fights this **curse**.
2. Maximizing variance and minimizing reconstruction error are the **same goal**.
3. The best directions solve **Cu = λu**. Each eigenvalue is the variance along its eigenvector.
4. $\tilde{\mathbf{X}} = \mathbf{U\Sigma V^T}$ gives the same directions without forming C, with $\lambda_i = \sigma_i^2/(n-1)$.
5. In practice: **scale** the features, fit on training data only, and choose k by variance or by cross-validation.
6. **Kernel PCA** handles curved structure through the kernel trick; tune the kernel and γ.
7. **t-SNE** draws faithful neighborhood maps for people to look at, but not for models to use.

## 8.3 Glossary

| term | meaning |
|---|---|
| centring | subtracting the mean so that the data is centered at the origin |
| covariance matrix C | d × d matrix of variances (diagonal) and covariances (off-diagonal) |
| eigenvector / eigenvalue | a direction u with Cu = λu; λ is the stretch factor (for C, the variance along u) |
| principal component (PC) | an eigenvector of C; a new axis built as a weighted sum of the original features |
| score | The coordinate of an example along a principal component, z = $u^T\tilde{x}$ |
| explained variance ratio | The share of the total variance kept by one component |
| SVD | X = UΣ$V^T$; V holds the principal components and $\sigma^2/(n-1)$ the variances |
| reconstruction error | mean squared distance between the original data and the data rebuilt from k components |
| kernel | a function k(x, x′) that equals a dot product in a lifted feature space |
| manifold | a lower-dimensional surface, possibly curved, on which the data lies |
| perplexity | t-SNE setting; roughly the effective number of neighbors of each point |
| KL divergence | a measure of how one probability distribution differs from another |

## 8.6 Further reading

- A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*, chapter on dimensionality reduction.
- L. van der Maaten and G. Hinton, "Visualizing Data using t-SNE", *Journal of Machine Learning Research*, 2008.
- scikit-learn user guide: "Decomposing signals in components" and "Manifold learning".